ShopSense

Customer Care & Order Operations Assistant

Sai Deepak (cs23btech11055) Harshith (cs23btech11045)

1. Background

E commerce applications generally receive a large number of customer support requests daily. Customers ask about order status, cancellations, returns, damaged products, and other issues. Most of the systems handle these using ticket queues and human support agents. Some rule based bots are also used for simple questions, but they generally stop when a decision is required.

Customer support systems these days are moving from FAQ chatbots to agentic systems that use retrieval to answer questions from company documents. So the next upgrade is an AI agent that can perform actions such as checking an order, cancelling it, creating a return, or requesting a refund.

Main difference here is, an agent is not only answering a question but also changing the state of the system. A wrong answer may frustrate a customer, but a wrong refund or order update can directly affect money and may be difficult to reverse. Therefore, a customer care agent needs to understand the request, check the current state of the order, follow company policy, and take correct action.

2. Problem Statement

2.1 Statement

The goal of ShopSense is to build a customer-care assistant that can decide and act correctly on customer requests using natural language, company policies, and store database data such as orders, products, payments, and returns, without requiring a human support agent to review every decision. The system should handle common order and refund operations while making sure that its actions follow the given policies. This is difficult because the same customer request can require different actions depending on the current state of the order and applicable policy. Existing evaluations of tool-using agents show that they can still make such mistakes and behave inconsistently across repeated runs. [1]

2.2 Scope

In scope

- Ticket triage for common customer requests.
- Order lookup, cancellation, return, and refund operations.
- Policy-based decisions with clause-level references.
- Customer memory across different conversations.

Out of scope

- Voice-based customer support.
- Multilingual conversations.
- Real payment gateways and real money movement.
- Machine-learning based fraud detection.
- Full production customer-support UI.
- Complete human handoff and escalation workflow.

2.3 Key Challenges

1. The same request can need a different action. A customer's message alone is often not enough to decide what to do. For example, suppose a customer says "cancel my order." If the order has not been shipped, cancellation may be allowed. If it has already been shipped, the system may

need to start a return instead. The agent therefore has to first check the order state and then apply the correct policy. The policies will be stored separately from agent and retrieved when required. The system should also record which policy clause was used for the decision so that the result can be checked later.

2. Some actions cannot be easily undone. Actions such as issuing a refund or changing an order are writes to the store system. A mistake in such an action can directly affect the customer. Because of this, write operations should have checks such as user confirmation, authorization, idempotency, and an audit log. A refund should not be executed twice just because the agent retries the same request.

3. Customers can make mistakes or give harmful instructions. A customer may misunderstand the order status, give incorrect information, or try to make the agent ignore its policies. The system should not assume that every statement from customer is correct. In some cases, refusing the request is correct outcome. Prompt injection is also important because the customer message itself is untrusted input. So design shpuld be something like keeping the component that **reads** customer message separate from the component that has permission to perform **write** operations. This follows the idea of separating untrusted input handling from privileged actions. [3]

4. The agent may not behave the same way every time. Even when the customer request and store state are unchanged, an LLM-based agent can sometimes choose different tools or different actions on different runs. This makes reliability an important part of the project. In τ-bench, strong function-calling agents succeeded on less than half of the tasks, and the retail domain had a pass^8 value below 25%, showing that getting one run correct does not mean the system is reliable across repeated runs. [1]

2.4 Assumptions

- The store database contains mock or synthetic customers, orders, products, and payments.
- Payment operations are simulated and do not move real money.
- The system supports English conversations only.
- The customer is assumed to be authenticated before using the assistant.

3. Existing Work and the Gap

Commercial platforms such as Sierra, Intercom Fin, Zendesk AI, and Salesforce Agentforce already provide AI systems for customer support, and in some cases these can perform actions such as managing orders or resolving support issues. [4] However, these are closed commercial systems, so their internal architecture and detailed reliability results are not available in enough detail for us to reproduce and compare them in a student project.

A closely related research benchmark is τ-bench. It evaluates an agent by simulating conversations between a user and an AI agent that has access to domain-specific tools and policy guidelines. The final result is checked by comparing the database state produced by the agent with an expected goal state instead of judging only the text response. It also introduces pass^k to measure how often an agent succeeds across repeated trials. τ^2-bench extends this type of evaluation to settings where both the user and the agent can take actions in the environment. [1, 2] Research on prompt-injection security also shows the need for **architectural controls** when agents are given access to tools. [3]

The gap we want to study is that these benchmarks mainly measure the reliability of a single agent with a flat tool list, while our project will also test whether a privilege-separated multi-agent architecture can improve that reliability. The component that reads untrusted customer text will not have write tools, so a prompt-injection instruction in the customer message will not directly have the ability to issue a refund.

4. Expected Output

4.1 The System

The expected output is a working prototype of ShopSense with the following components:

- A seeded store database containing customers, products, orders, returns, refunds, and payment information.
- Read and write operations exposed as tools through an MCP server.
- A LangGraph pipeline that orchestrates the stages of the main flow described below.
- A policy RAG component that retrieves relevant policy clauses and returns the clause used for each decision.
- Customer memory that stores useful information from previous conversations.
- A refund ceiling that is checked in application code before a refund operation is allowed.
- Privilege separation so that the component processing raw customer text cannot directly call write tools.

The main flow will therefore be: customer request → triage → retrieve policy → make a plan → verify the plan → ask for confirmation when required → execute the action → store an audit record.

4.2 Evaluation and Metrics

A scenario-based evaluation suite of around 25–40 cases will be created. It will include normal requests, ambiguous requests, cases where the correct outcome is refusal, and cases containing prompt-injection attempts. The proposed system will be compared with a simpler single-agent baseline using the same store data and tools.

The main metrics will be:

- **Task success:** whether the final store/database state matches the expected state.
- **Policy-violation rate:** percentage of cases where the agent performs an action that is not allowed by policy.
- **Injection-resistance rate:** percentage of injection cases where no unauthorized action is taken.
- pass^k**:** success rate when the same scenario is executed repeatedly.
- **Tool-call efficiency:** number of tool calls required to complete a successful task.

The intended outcome here is measured comparison showing whether privilege separation and explicit policy grounding improve task success, policy compliance, and run to run consistency over a single agent baseline.

4.3 Limitations

The items listed as out of scope in Section 2.2 are deferred because they would increase the size of the project without directly testing the main research question. The evaluation will also use an LLM-based simulated user rather than real customers. Therefore, the measured results may not fully represent real customer behaviour and should be treated as an upper bound on how the system may perform in practice.

References

[1] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," arXiv:2406.12045, 2024.

[2] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan, "τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment," arXiv:2506.07982, 2025.

[3] L. Beurer-Kellner et al., "Design Patterns for Securing LLM Agents against Prompt Injections," arXiv:2506.08837, 2025.

[4] Sierra, Intercom Fin, Zendesk AI, and Salesforce Agentforce, official product documentation and websites, accessed September 2026.